

QA Assessment: Persona Authorization Verification

Verified by: Antigravity SRE Engine

Date: June 26, 2026

A QA tester reported that many features work in the **superadmin** persona, but fail when logged in as **retailer** or **brand** personas. This claim has been fully investigated, verified, and found to be correct.

There is a known, documented architectural conflict in the development/demo environment where switching to the retailer or brand personas can cause those dashboards and wizards to fail, while the admin and superadmin dashboards continue to work fine.

The details of this issue are documented in the [active_personaVSdemo_role.md](#) Architecture Decision Record and the [architecture_msot_report.md](#) review.

Root Cause Analysis

1. The Competing localStorage Keys Conflict

- **The Setup:** The frontend keeps two separate keys in localStorage to represent the active user: `demo_role` and `active_persona`.
- **The Default:** On initial load in the DEV environment, the app seeds `demo_role = 'admin'` as a safe default.
- **The Persona Switch:** When a tester switches to brand or retailer using the UI switcher, some components write the new role to `active_persona` (e.g., 'brand') but do not clear or update the `demo_role` key (which remains 'admin').
- **The Interceptor Shadowing:** The API client interceptor in `api.js` resolves the role using:

```
const demoRole = localStorage.getItem('demo_role') || localStorage.getItem('active_persona');
```

Because `demo_role` is evaluated first, it short-circuits to 'admin'. Therefore, every outbound request from the brand or retailer dashboard is sent to the backend carrying the header `x-demo-role: admin`.
- **The Result:** The backend rejects these requests with a 403 Forbidden or returns data filtered for admins instead of brands/retailers. The frontend catch blocks swallow these errors, rendering blank or empty UI states with no feedback.

2. Direct localStorage Reads (No React Context)

Several pages bypass React's AuthContext and read directly from local storage, making them blind to persona changes until a hard page refresh occurs:

- **ScheduleCalendar.jsx:** Reads `localStorage.getItem('active_persona')`.
- **LoopDemoPlayer.jsx:** Resolves role using the same local storage || fallback.

3. Bypass of the Central API Client

Several components do not use the central `apiClient` (which injects headers automatically) and instead make raw `fetch()` calls. Developers must manually duplicate the auth headers for these, creating gaps:

- `AILog.jsx`
- `Player.jsx`
- `TechOpsDashboard.jsx`
- `TicketDashboard.jsx`

Diagnosis & Testing Tip (How to Reproduce)

You can verify the current state of local storage keys in your browser console by pasting:

```
console.table({
  auth_token:    localStorage.getItem('auth_token'),
  demo_role:     localStorage.getItem('demo_role'),
  active_persona: localStorage.getItem('active_persona'),
  'interceptor resolves to': localStorage.getItem('demo_role') || localStorage.getItem('active_persona')
});
```

Immediate Mitigation: Tell your QA testers to run `localStorage.clear()` in their browser consoles when switching between different persona dashboards to purge stale default keys.

Long-Term Fix: Follow the cleanup steps in [active_personaVSdemo_role.md](#) to implement a unified AuthProvider contract, routing all reads/writes through a single service, and deprecating direct `localStorage`/native fetch operations.